

Lesson 4 — Sprites And Spritesheets

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The three forms of drawImage

```
drawImage(img, dx, dy);
drawImage(img, dx, dy, dw, dh);
drawImage(img, sx, sy, sw, sh,
           dx, dy, dw, dh);
```

Only the **nine-argument** form can take a piece of a sheet. Using the four-argument one with a sheet puts the whole sheet on screen — the commonest bug in this lesson.

```
s = SOURCE      (where to copy FROM)
d = DESTINATION (where to put it)
```

Which cell of the grid

```
const sx = column * FRAME_W;
const sy = row   * FRAME_H;
```

The frame timer

```
sprite.timer += dt;
while (sprite.timer >= FRAME_TIME) {
  sprite.timer -= FRAME_TIME;
  sprite.frame = (sprite.frame + 1) %
    FRAMES;
}
```

- **while**, not **if** — a slow frame may owe you two poses.
- **subtract**, do not zero — zeroing loses the leftover and drifts slow.
- **%** wraps the counter with no **if**.

State chooses the animation

```
const ANIMATIONS = {
  idle: { row: 0, frames: 2, fps: 2 },
  walk: { row: 1, frames: 6, fps: 8 }
};

if (state !== lastState) {
  frame = 0; timer = 0; lastState = state;
}
```

Compare `state !== lastState`, not `state === "walk"`: you want the moment it **became** walk.

Facing left

```
ctx.save();
ctx.translate(x, y);
ctx.scale(-1, 1);
ctx.translate(-x, -y);
ctx.drawImage(/* ... */);
ctx.restore();
```

The two translates are because the mirror is about **x = 0**, not about the sprite.

Crisp pixels

```
ctx.imageSmoothingEnabled = false; // in
draw()
const drawX = Math.round(pos.x); //
round when DRAWING, not when moving
```

```
canvas { image-rendering: pixelated; }
```

Loading

```
const img = new Image();
img.onload = () => ready = true;
img.src = "hero.png"; // src LAST
```

Drawing before it loads draws **nothing**, with no error.

A canvas is an image

```
const sheet =  
document.createElement("canvas");  
// draw into sheet.getContext("2d") ...  
ctx.drawImage(sheet, ...);    // works exactly  
like a PNG
```

Section A — Recall

A1. Give three reasons real games put many poses in one image instead of one file per pose. [3 marks]

A2. In `drawImage(img, sx, sy, sw, sh, dx, dy, dw, dh)`, which four numbers describe the image [2 marks] and which four describe the canvas?

A3. Why must the animation have its own timer rather than advancing one pose per game frame? [3 marks]
Name two separate problems with the one-per-frame version.

A4. Why `while` rather than `if` in the frame timer? [2 marks]

A5. What does `(frame + 1) % FRAMES` do, and what does it save you writing? [2 marks]

A6. Why should the character's *state* choose the animation, rather than the key being pressed? Give a [3 marks]
practical benefit.

A7. You round the position when drawing but not when moving. Why both halves of that? [2 marks]

Section B — Predict the output

B1. Frames are 32×32 and the sheet has 6 columns and 3 rows. What are `sx` and `sy` for row 2, column [3 marks]
4? What is the whole sheet's size in pixels?

B2. The game runs at 60 fps and `FRAME_TIME` is $1/8$. How many game frames pass per pose? How [3 marks]
many poses in two seconds?

B3. What appears on screen, and why?

[4 marks]

```
// sheet is 192 x 32, frames are 32 x 32
ctx.drawImage(sheet, 100, 100, 32, 32);
```

B4. `FRAMES` is 6 and the current frame is 5. The `% FRAMES` has been deleted. What is `sx` next frame, and what is drawn? [3 marks]

B5. `FRAME_TIME` is $1/8 = 0.125$ s and the game runs at exactly 60 fps, so `dt` is 0.01667. The author wrote `timer = 0` instead of `timer -= FRAME_TIME`. Work out the real time each pose is held for, and how far behind the animation is after one minute. [4 marks]

Section C — Find and fix the bug

C1. Nothing is drawn, and there is no error in the console. Name the most likely cause and a way to prove it. [3 marks]

```
const img = new Image();
img.src = "hero.png";
ctx.drawImage(img, 0, 0, 32, 32, 100, 100, 32, 32);
```

C2. The walk animation never gets past its first pose, however long the player walks. [3 marks]

```
if (player.state === "walk") {
    player.frame = 0;
    player.timer = 0;
}
player.timer += dt;
while (player.timer >= FRAME_TIME) { /* ... */ }
```

C3. The character disappears completely whenever they turn to face left. Explain and fix.

[4 marks]

```
ctx.save();
if (facing === -1) { ctx.scale(-1, 1); }
ctx.drawImage(sheet, sx, 0, FW, FH, x, y, FW, FH);
ctx.restore();
```

C4. The sprite is sharp when standing still and blurry while walking slowly. Smoothing is already off. What is left?

[3 marks]

C5. On a slow laptop the walk cycle plays in slow motion, while the character still moves at the right speed. Both bugs are in these four lines — name them.

[4 marks]

```
player.timer += 1;
if (player.timer >= 8) {
  player.timer = 0;
  player.frame = player.frame + 1;
}
```

Section D — Build it

Start from your lesson 3 ship, or from `code/01-make-a-spritesheet.html`.

- ☐ **Checkpoint 1.** Build a spritesheet in code: an offscreen canvas, 6 cells across, with a figure in each whose legs are at different angles. Draw the finished sheet on screen at 3× size and look at it.
- ☐ **Checkpoint 2.** Draw *one* cell of it with the nine-argument `drawImage`. Add two keys that step the frame index up and down by hand, and watch `sx` change.
- ☐ **Checkpoint 3.** Add the frame timer so it animates by itself. Put the pose number and the timer on screen while you work — you cannot debug what you cannot see.
- ☐ **Checkpoint 4.** Add a second row for “idle” (two slow poses) and an `ANIMATIONS` table. Make the character's *state* pick the row, and reset the frame when the state changes.
- ☐ **Checkpoint 5.** Make them face the way they are moving, using `scale(-1, 1)` and the two translates. Test by walking left, right, then stopping — they should keep facing the way they last went.
- ☐ **Checkpoint 6.** Turn smoothing off and round the draw position. Compare a sprite drawn at `x` with one drawn at `Math.round(x)`, side by side, moving very slowly.
- ☐ **Checkpoint 7.** Add a switch that advances one pose per game frame, so you can show somebody the difference. Keep it; it is the best demonstration in the lesson.

IF NOTHING APPEARS

Draw a bright filled rectangle at exactly the same destination coordinates first. If the rectangle appears and the sprite does not, the problem is the *source* rectangle or the image not being ready — not the position. That one test saves a lot of time.</p>

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning, and on naming the cost.

E1. A sprite is drawn from a point: top-left, centre, or middle of the feet. Choose one for a platform game and justify it. What happens when a character's picture gets taller? What is the collision box measured from?

E2. A player moving at double speed still shows 8 poses a second, so their feet slide. How would you tie animation speed to actual speed? What breaks if you tie it too tightly — think about a character being pushed, or standing on a moving platform.

E3. Design the state machine for a character who can be hurt while walking, jumping or standing still. Does "hurt" interrupt, or play on top? What happens if they are hurt twice in quick succession?

E4. The hard one. Your sheet has 200 poses and you need team-red and team-blue shirts. Three real options: ship two sheets, recolour once at load time, or draw a separate shirt layer on top. Pick one, name its cost, and say what would make you change your mind.

Stretch goals

1. Animation fps proportional to actual speed, so the feet never slide.
2. Recolour your sheet at load time into two team colours.
3. A ping-pong animation: 0,1,2,3,2,1 — which % alone cannot do.
4. Write an atlas packer: a list of offscreen canvases in, one sheet plus a lookup of source rectangles out.
5. Onion skin: draw the previous two poses faintly behind the current one.